CS 214: Data Structures

Search and Analysis of Algorithms

Slide contents follow

Kruse and Leung "Data Structures & Program Design in C"

Prepared by:

Waleed A. Yousef, Ph.D.

© Waleed A. Yousef 2008

Motivation: Why Search?

- To find things ☺

- We will be concerned only with internal searching in the memory, i.e., in data structures not in external searching in secondary memory, i.e., in files. Nevertheless, there is a strong relationship.

- Of course we assume that each element has a key to be used for searching. To have our code as general as possible we do not write the code for particular data type.

- **Always, think generally before you start coding; this is to minimize upgrading and modifying your code later on.**

- Our purpose is to study data structures not algorithms, but we want to see the deep connection between both through studying sequential and binary search.

- Let us see the implementation in C.

Arabsera

© Waleed A. Yousef 2008

```
typedef int KeyType;

#define EQ(a, b)  ((a)==(b))
#define LT(a, b)  ((a)<(b))
#define LE(a, b)  ((a)<=(b))
#define GT(a, b)  ((a)>(b))
#define GE(a, b)  ((a)>=(b))
//#define EQ(a, b)  (!strcmp((a), (b)))
//#define LT(a, b)  (strcmp((a), (b))<0)
//#define LE(a, b)  (strcmp((a), (b))<=0)
//#define GT(a, b)  (strcmp((a), (b))>0)
//#define GE(a, b)  (strcmp((a), (b))>=0)
typedef struct elementtype{
    KeyType  key;
    int      year;
    int      etype;
union{
       int   year;
       float age;
       ...
    }info;
}ElementType;
```

These many parentheses to ensure that the precedence rule will not change the order o expression. E.g., how `EQ(1||0, 3)` will be evaluated without parentheses? (== is higher in precedence than ||)

`KeyType` will be either, `int`, `float`, `char`, or string.

When we search we will compare, whether equal , less, or larger. For ultimate generality define these relations in macros. Functions are slower.

This code is general enough for our purpose.

© Waleed A. Yousef 2008

```
/*Pre: list is created.
Post:return the location if element exists O.W return -1
This code is implementation-independent */

int SequentialSearch(KeyType target, List *pl){
    int current, s=ListSize(pl);
    ListEntry currententry;
    for(current=0; current<s; current++){
        RetrieveList(current, &currententry, pl);
        if(EQ(target, currententry.key))
            return current;
    }
    return -1;
}
```

See, how we did not call ListSize here so that we save the call time every iteration.

if `current position` trick is not used in linked implementation this statement alone is $O(n)$, which is very inefficient, because every time `RetrieveList` is called the elements are traversed from the first one.

Re-write the code above in the implementation level for linked and contiguous implementations to save the time of the function calls. (compare to the book).

What about special cases in the above code?



© Waleed A. Yousef 2008

Analysis of Algorithm

Definition: The behavior of a key-comparison (because there are other searching techniques) searching algorithm is measured in number of key comparisons.

Sequential search: From the previous algorithm it is obvious that if the element is in position i, the number of comparisons k is given by:

$$k(i)=i+1$$

k in any algorithm is a random variable whose mean (average) depends on the distribution of the data. The mean is given by:

$$E(k)=\sum_{i=0}^{n-1} k_i \Pr(k_i)$$

Then, for any algorithm we have to assume a probability distribution for the data. This is a field by itself known as probabilistic analysis of algorithms. For easier analysis we usually assume uniform distribution, i.e.,

$$\Pr(k_i)=1/n$$

This is what is called in book "on average": they means on average **with uniform dist.**

© Wael A. Yousef 2018

$$E(k) = \sum_{i=0}^{n-1} k_i \, Pr(k_i) = \frac{1}{n} \sum_{i=0}^{n-1} k_i = \frac{1}{n} \sum_{i=0}^{n-1} (i+1) = \frac{1}{n} \left(\frac{n(n-1)}{2} + n \right) = \frac{1}{2} n + \frac{1}{2}$$

The worst case scenario is when the search is unsuccessful or the element is in last entry. In this case the number of comparisons is n

The best case is when the element is in the first position, then the number of comparison is 1.

For unsuccessful search:
There is only one case of unsuccessful search that takes n comparisons.

© Waleed A. Yousef 2008

Binary Search

Can we speed up the search time? We will assume that we will search in an ordered list.

Definition: An ordered list is a list in which each entry contains a key, such that the keys are in order. That is, if entry i comes before entry j in the list, then the key of entry i is less than or equal to the key of entry j.

This requires replacing the `InsertList` with `InsertOrder`. See the connection between enhancing the algorithms and designing the data structures.

We will consider only the second implementation of Binary search, i.e., `Binary2Search`. Read the first version from the book.

Let us first write `InsertOrder`

© Waleed A. Yousef 2008

```
/*Pre: created, not full, and ordered.
post:  e inserted in order. if the new element has a key equal to
an element in the list it will be inserted before it*/

void InsertOrder(ListEntry e, List *pl){
   int current, s=ListSize(pl);
   ListEntry currententry;
   for(current=0; current<s; current++){
      RetrieveList(current, &currententry, pl);
         if (LE(e.key, currententry.key))
            break;
      }
   InsertList(current, e, pl);
}
```

E.g., 8 will be inserted in position 3

if `current position` trick is not used in linked implementation this statement alone is $O(n)$, which is very inefficient, because every time `RetrieveList` is called the elements are traversed from the first one.

Re-write the code above in the implementation level for contiguous implementations to save the time of the function calls.

What about special cases in the above code?



© Waleed A. Yousef 2008

Binary Search

The following representation is from Knuth "*The art of computer programming*" (Sec. 6.2.1):

Searching for 653

[061	087	154	170	275	426	503	509	512	612	653	677	703	765	897	908]
061	087	154	170	275	426	503	509	[512	612	653	677	703	765	897	908]
061	087	154	170	275	426	503	509	[512	612	653]	677	703	765	897	908
061	087	154	170	275	426	503	509	512	612	[653]	677	703	765	897	908

Searching for 400

[061	087	154	170	275	426	503	509	512	612	653	677	703	765	897	908]
[061	087	154	170	275	426	503]	509	512	612	653	677	703	765	897	908
061	087	154	170	[275	426	503]	509	512	612	653	677	703	765	897	908
061	087	154	170	[275]	426	503	509	512	612	653	677	703	765	897	908
061	087	154	170	275]	[426	503	509	512	612	653	677	703	765	897	908

It looks recursive; let us try it. The interface has to be:

```
int RecBinary2Search(KeyType, List *)
```

However, it seems from the table that we need to specify the start and the end indices. Therefore, we have to write another recursive function and call it in the above.

© Waleed A. Yousef 2008

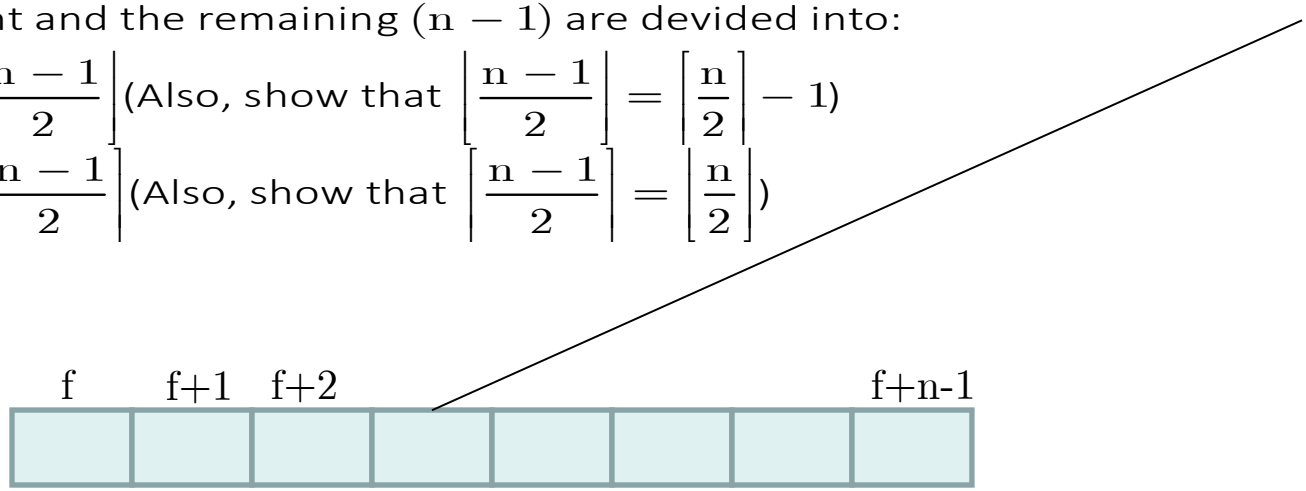
Binary Search

In general we split at: $\text{Mid} = \left\lfloor \frac{\text{first} + \text{last}}{2} \right\rfloor = \left\lfloor \frac{f + (f + n - 1)}{2} \right\rfloor = \left\lfloor f + \frac{n-1}{2} \right\rfloor = f + \left\lfloor \frac{n-1}{2} \right\rfloor$

We split at one element and the remaining $(n-1)$ are devided into:

elements at the LT = $\left\lfloor \frac{n-1}{2} \right\rfloor$ (Also, show that $\left\lfloor \frac{n-1}{2} \right\rfloor = \left\lceil \frac{n}{2} \right\rceil - 1$)

elements at the RT = $\left\lceil \frac{n-1}{2} \right\rceil$ (Also, show that $\left\lceil \frac{n-1}{2} \right\rceil = \left\lfloor \frac{n}{2} \right\rfloor$)



It looks recursive; let us try it. The interface has to be:

```
int RecBinary2Search(KeyType, List *)
```

However, it seems from the table that we need to specify the start and the end indices. Therefore, we have to write another recursive function and call it in the above.

© Waleed A. Yousef 2008

```
/*pre: list is ordered  Post: location returned, O.W. -1*/
int RecBinary2(List *pl, KeyType k, int bottom, int top){
    int middle;
    if (bottom<=top){
        middle=(bottom+top)/2;
        if (EQ(k, pl->entry[middle].key))
            return middle;
        if (LT(k, pl->entry[middle].key))
            return RecBinary2(pl, k, bottom, middle-1);
        else
            return RecBinary2(pl, k, middle+1, top);
    }
    return -1;//Base condition
}
int RecBinary2Search(KeyType k, List *pl){
        return RecBinary2(pl, k, 0, pl->size-1);
}
```

[061	087	154	170	275	426	503	509	512	612	653	677	703	765	897	908]
061	087	154	170	275	426	503	509	[512	612	653	677	703	765	897	908]
061	087	154	170	275	426	503	509	[512	612	653]	677	703	765	897	908

© Waleed A. Yousef 2008

```
/*pre: list is ordered  Post: location returned, O.W. -1*/
int Binary2Search(KeyType k, List *pl){
    int middle, bottom=0, top=pl->size-1;
    while(bottom<=top){
        middle=(bottom+top)/2;
        if (EQ(k, pl->entry[middle].key))
            return middle;
        if (LT(k, pl->entry[middle].key))
            top=middle-1;
        else
            bottom=middle+1;
    }
    return -1;
}
```

[061	087	154	170	275	426	503	509	512	612	653	677	703	765	897	908]
061	087	154	170	275	426	503	509	[512	612	653	677	703	765	897	908]
061	087	154	170	275	426	503	509	[512	612	653]	677	703	765	897	908

© Waleed A. Yousef 2008

Comparison Side-by-Side

```
int Binary2Search(KeyType k, List *pl){
   int middle, bottom=0, top=pl->size-1;
   while(bottom<=top){
        middle=(bottom+top)/2;
        if (EQ(k, pl->entry[middle].key))
           return middle;
        if (LT(k, pl->entry[middle].key))
           top=middle-1;
        else
           bottom=middle+1;
   }
   return -1;
}

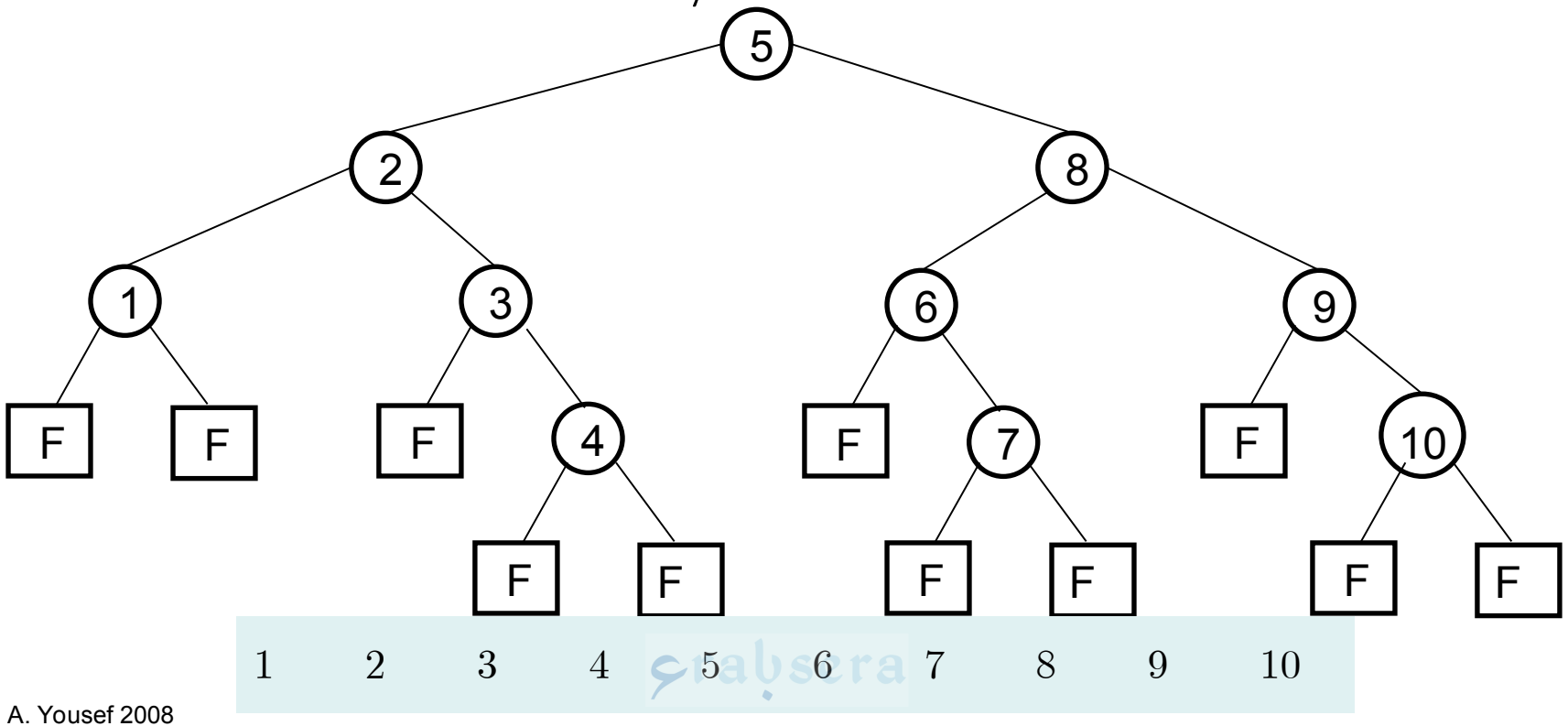
int RecBinary2(List *pl, KeyType k, int bottom, int top){
   int middle;
   if (bottom<=top){
        middle=(bottom+top)/2;
        if (EQ(target, pl->entry[middle].key))
          return middle;
        if (LT(k, pl->entry[middle].key))
           return RecBinary2(pl, k, bottom, middle-1);
        else
           return RecBinary2(pl, k, middle+1, top);
   }
   return -1;
}
```

© Waleed A. Yousef 2008

Another important connection between algorithms and data structures is this. Binary Search is suitable only for the contiguous implementation. However, if we have to place the data in a linked structures (linked list) and be able in the same time to fasten the search what should we do?

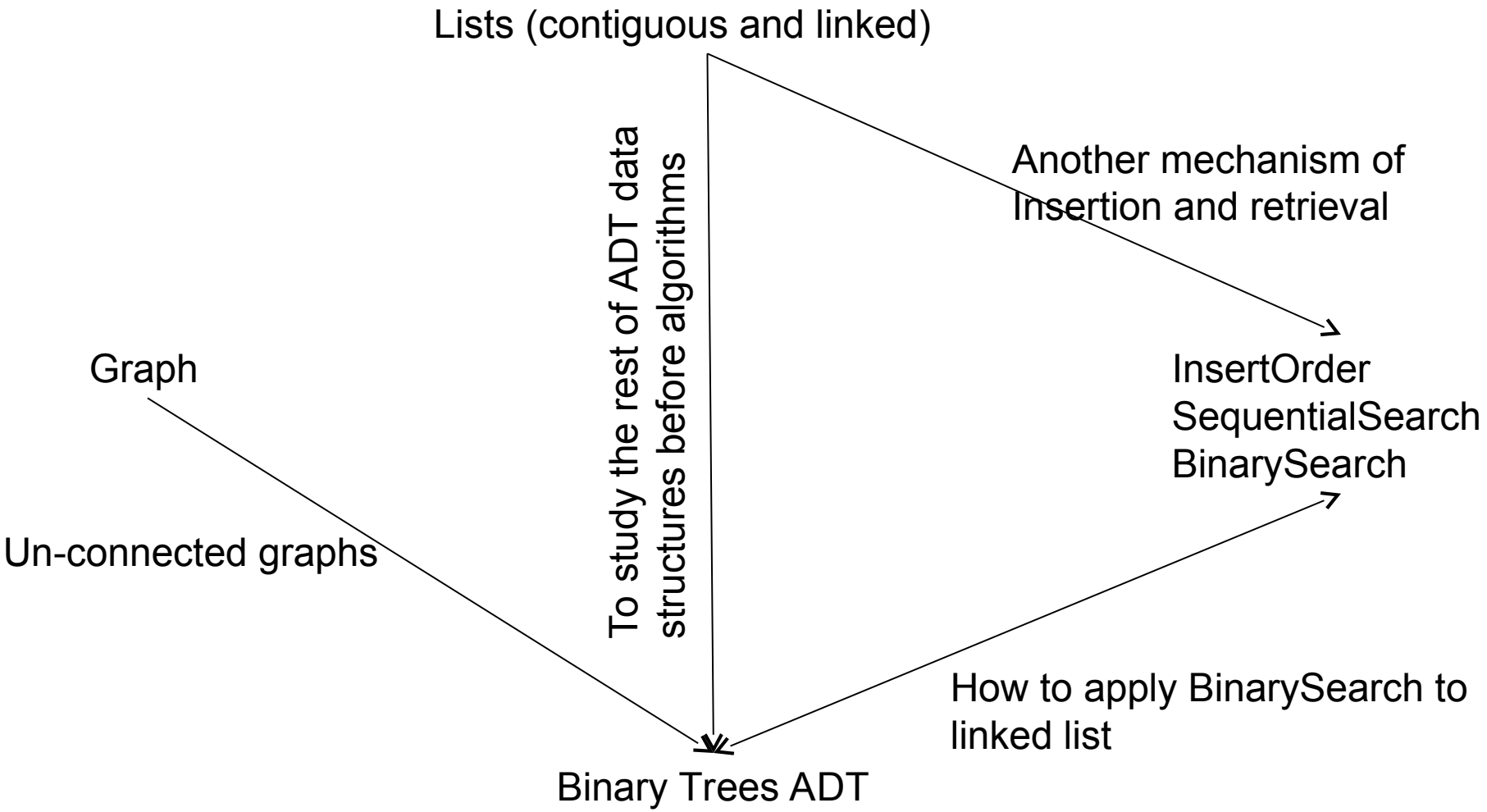
This will be achieved by implementing the linked list as a binary tree as we will see later (Ch. 9). To see the connection let us see first how we analyze binary search. The idea comes from analyzing the binary search for contiguous implementation.

The analysis of Binary Search requires basic definitions and mathematical relations for trees as a mathematical structure NOT yet an ADT data structure.



© Waleed A. Yousef 2008

How the different topics are related to each other (Different order of study)



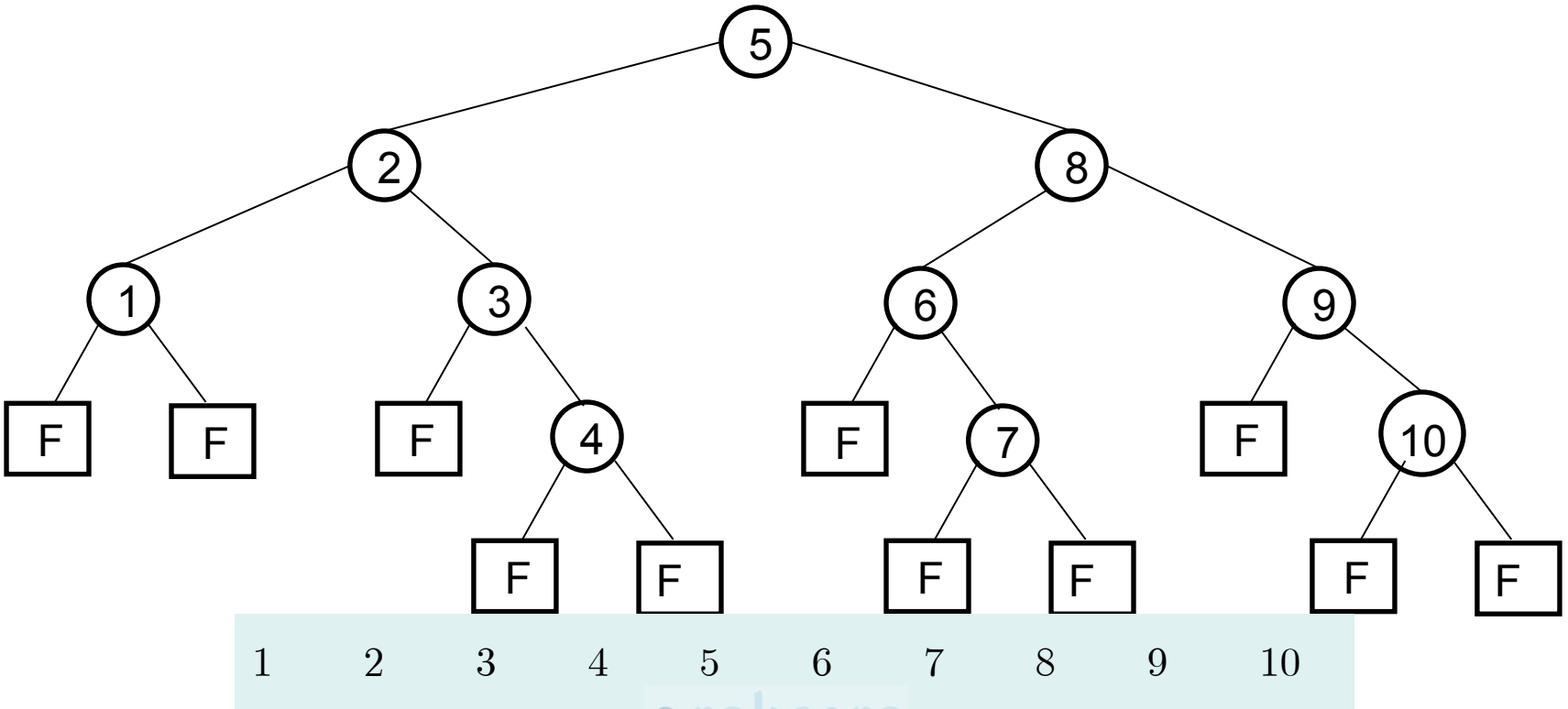
© Waleed A. Yousef 2008

Analysis of Binary Search Algorithm

Main references of the following material are:

Knuth, D. E. (1997). The art of computer programming. Reading, Mass., Addison-Wesley.

Mahmoud, H. M. (2000). Sorting : a distribution theory. New York, John Wiley & Sons.

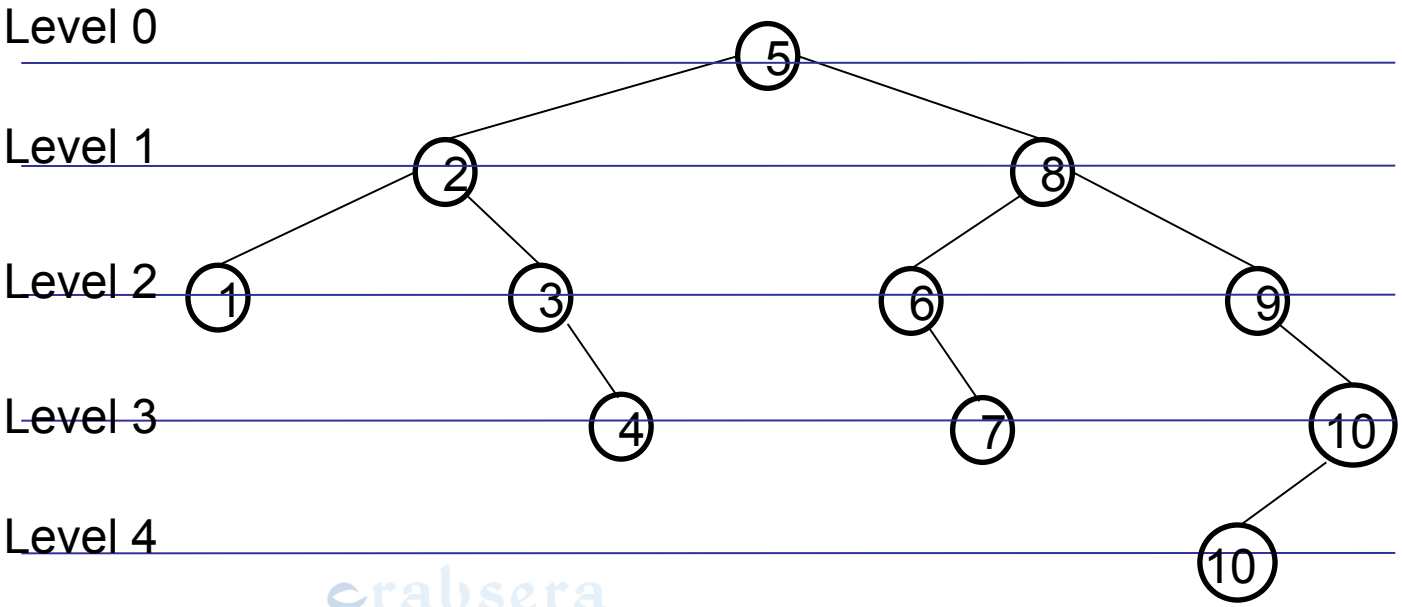


© Waleed A. Yousef 2008

Basic definitions of Binary Trees (will help also for Ch. 9)

Definition: A binary tree is either empty, or it consists of a **node** (vertex) called the root together with two binary trees called the left subtree and the right subtree of the root.

- The only node at level 0 is the root.
- A node may have up to two children in the next level.
- The children of a node are joined to their parents by links called edges.
- The shown tree is a general BT, not a BT produced by binary searching.



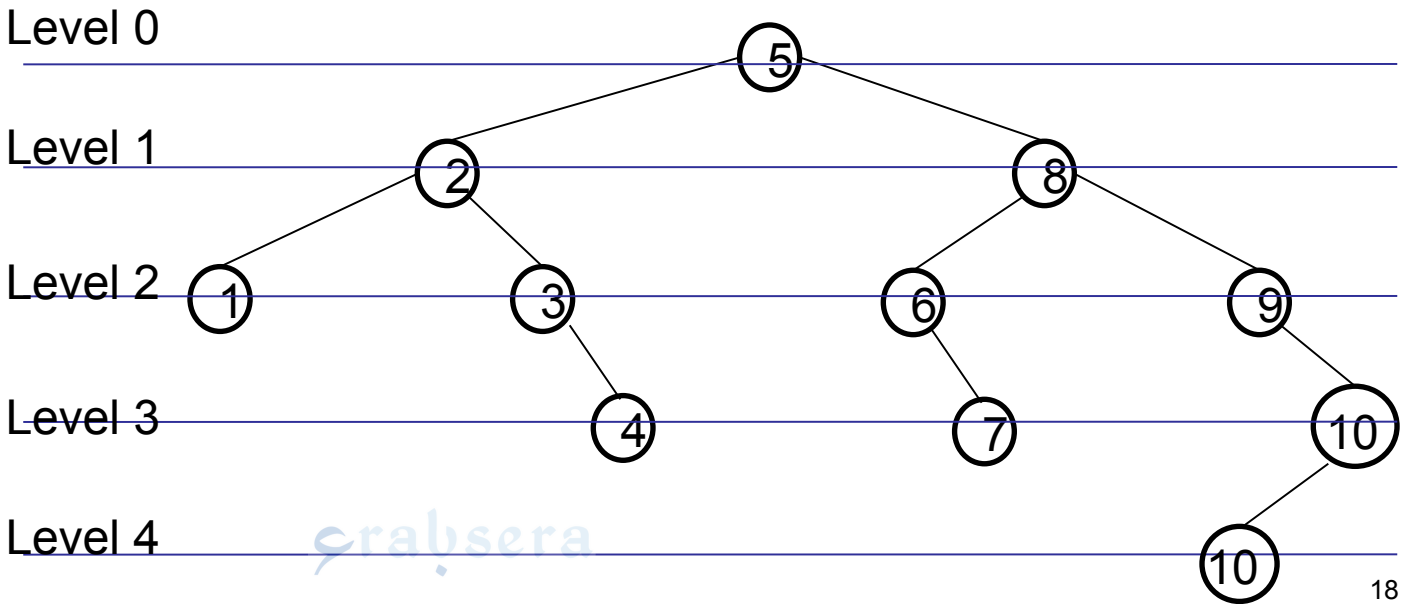
© Waleed A. Yousef 2008

Definition: The depth (or height) of a node in the tree is the node's distance from the root; i.e., its level.

Equivalent Definition (recursive): The depth of a node is 1 + the depth of its parent. And the root is at depth 0.

The collection of nodes in the subtree rooted at a node (excluding the node itself) is referred to as its **descendants**. E.g., descendants of 8 are: 6, 7, 9, 10, 10.

The collection of nodes encountered by climbing down (to the root) a path from a node to the root of the whole tree is the node's **ancestors** (or **predecessors**). E.g., ancestors of 3 are: 2, 5.



© Waleed A. Yousef 2008

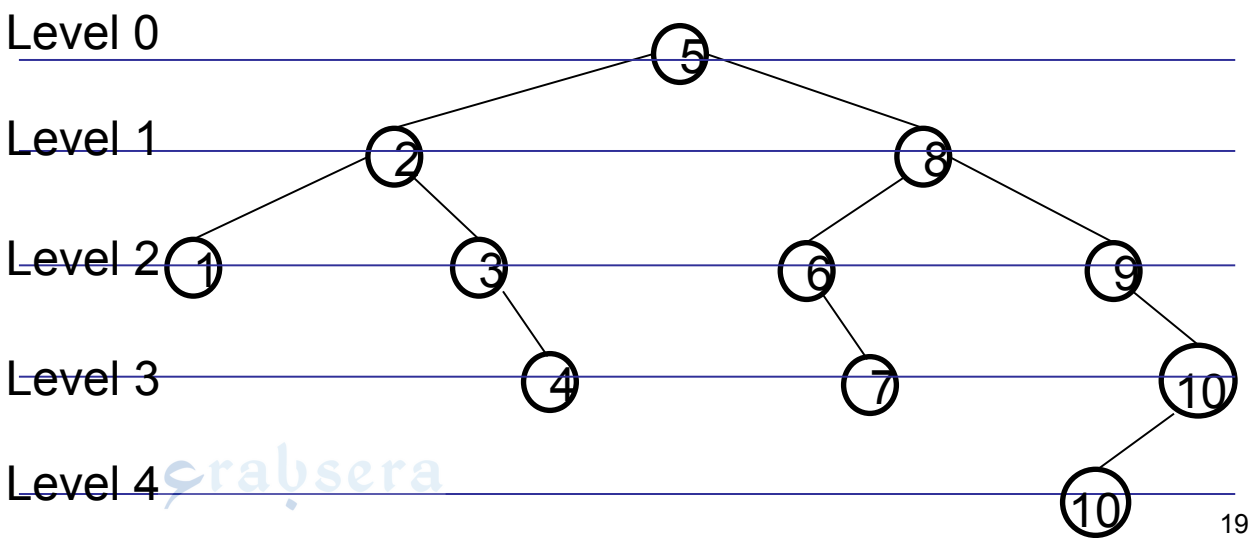
The depth or height h of a tree: is the maximum height among nodes. Also, we can say

$$\mathrm{h} = \begin{cases} 0 & \text{tree has only root} \\ 1 + \max(\mathrm{h}(\text{left tree}), \mathrm{h}(\text{right tree})) \end{cases}$$

Outdegree: is the number of edges coming out of a node. In binary trees this is at most 2.

Indegree: is the number of edges coming to a node. In any tree this is 1.

Lemma: a saturated (with a maximum number of nodes) level l has 2^{l} nodes.
Proof: For $\mathrm{l} = 0$ the statement is true. Assume it is true for some k, then the number of nodes in this level is 2^{k}. Then, the maximum number of nodes of next level, $\mathrm{k}+1$, is 2 x $2^{\mathrm{k}} = 2^{\mathrm{k}+1}$, which completes the induction. ■



© Waleed A. Yousef 2008

Saturated Level: a level with maximum number of nodes.
Full Tree: a tree whose all levels are saturated

Corollary : If all levels (from 0 to h) are saturated then the tree has $2^{h+1}-1$ nodes.

Proof :

$$n = \sum_{l=0}^{h} 2^l = 2^{h+1} - 1$$ ■

Lemma : The height h_n of a binary tree on n nodes satisfies:

$$\lceil \lg(n+1) \rceil - 1 \le h_n \le n - 1.$$

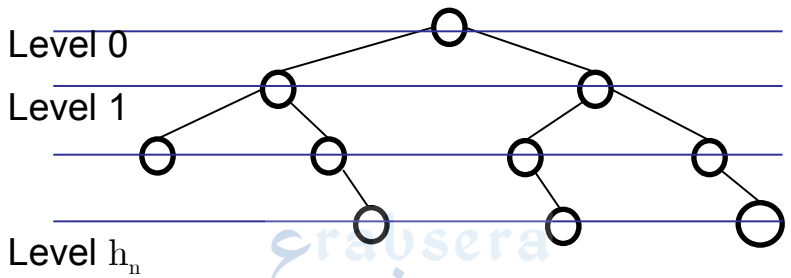
Proof : The upper bound is trivial. We have from the corollary above:

$$n \le 2^{h_n+1} - 1, \text{ then}$$
$$h_n \ge \lg(n+1) - 1$$

But h_n is integer; therefore

$$h_n \ge \lceil \lg(n+1) - 1 \rceil$$
$$= \lceil \lg(n+1) \rceil - 1$$ ■

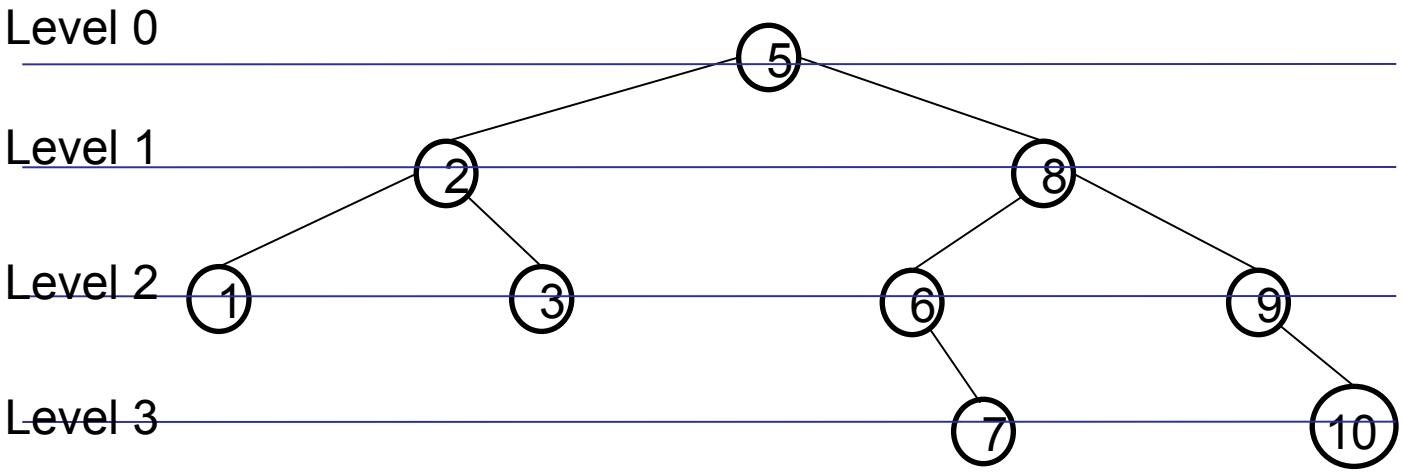
We will show soon that: $\lceil \lg(n+1) \rceil - 1 = \lfloor \lg(n) \rfloor$



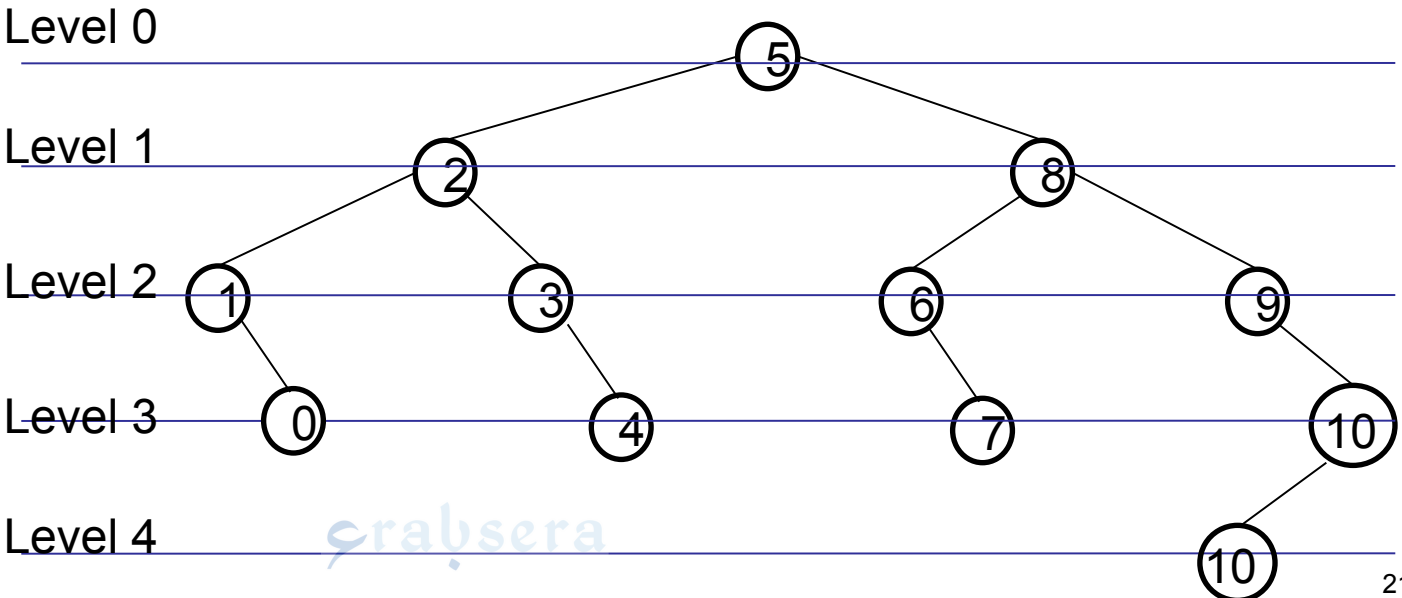
© Waleed A. Yousef 2008

Complete Tree : is the tree whose all levels are saturated except possibly the last one.
Complete Tree $\rightarrow$ leaves are at the last two levels at most (the converse is not true)
Complete Tree $\leftrightarrow$ it is the shortest tree for given number of nodes.

Complete



Not Complete



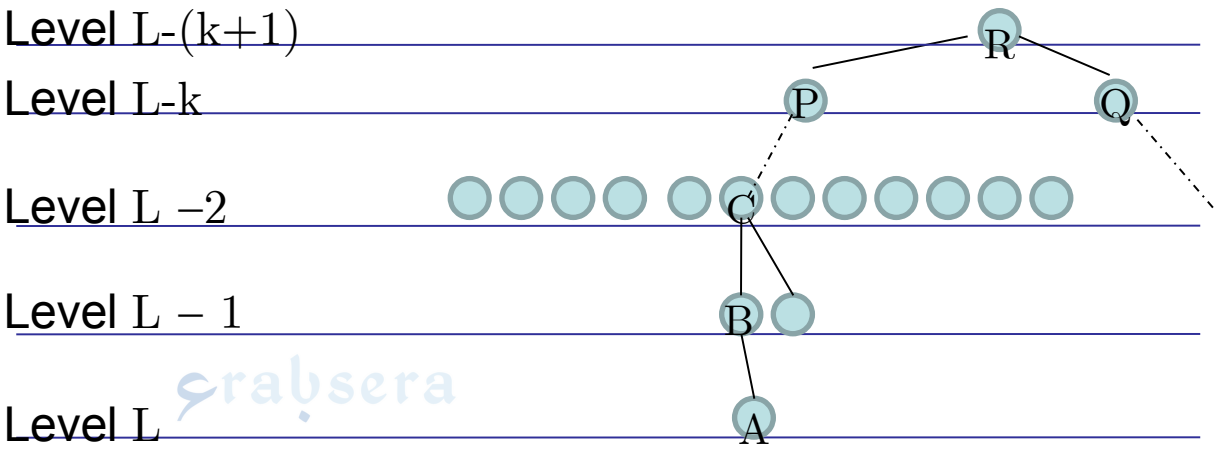
© Waleed A. Yousef 2008

Lemma: the tree produced by the binary search is complete.

Proof: First notice that every split produces two subtrees which differ in size by at most one node. We will show, by induction, that, any node at level $L - k$, $k \geq 2$, (because for $k=1$ is trivial) having a path to a node at the last level L will have complete subtree. Hence, the root, which has path to any node at last level L, has a complete tree. Consider a node A at the last level L

Basis step: For $k = 2$, C must have another child than B to have a valid split at C.

Induction step: Assume the statement is true for some k; then P has a complete tree. The parent R must have another child Q, so that the split is valid. If Q has any child at L, then it must have a complete tree (by the induction hypothesis, since it is at the level $L - k$). If Q does not have any child at L, then it must have, as well, complete tree so that the split at R produces two trees different in size by just one node; namely A. For both cases, it turns out that R, which is the only node at level L-$(k+1)$ having path to A, has a complete subtree. Then the statement is true for $k+1$. Hence, the statement is true for all k. Therefore, for $k = L$, where $(L - k = 0)$ the statement is true for the root. Hence, the root has a complete tree. ■



© Waleed A. Yousef 2008

Theorem: For the binary search algorithm:

1) the best case takes 1 comparison.

2) the worst successful case takes $h_n + 1 = \lceil \lg(n+1) \rceil = \lfloor \lg(n) \rfloor + 1$ comparisons

3) the unsuccessful case takes exactly $h_n + 1$ if the tree is full, i.e., all levels are saturated and takes either h_n or $h_n + 1$ otherwise.

4) no other search method (based on comparison) does better than binary search in the worst case.

Proof:

1)is trivial.

2)We need $h_n + 1$ comparisons for worst case. For any binary tree $\lceil \lg(n+1) \rceil - 1 \le h_n$. Since our tree is complete, $h_n + 1 = \lceil \lg (n+1) \rceil$, which is equal to $\lfloor \lg(n) \rfloor + 1$ by the following:

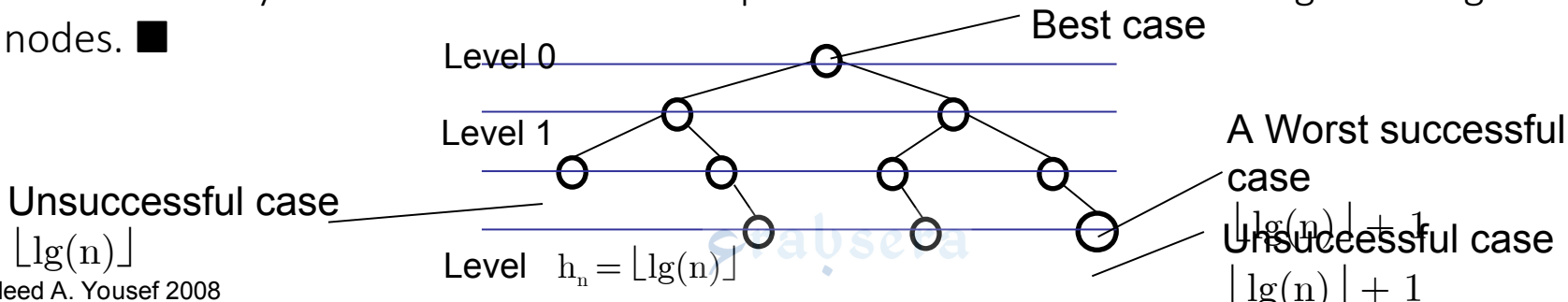
$$(\#\text{ nodes up to level h-1}) = 2^h - 1 < 2^h \le n < n+1 \le 2^{h+1}$$

$$h \le \lg(n) < \lg(n+1) \le h+1$$

$$h = \lfloor \lg(n) \rfloor = \lceil \lg(n+1) \rceil - 1$$

3)Is obvious. The number of comparisons is either $h_n + 1 = \lceil \lg(n+1) \rceil = \lfloor \lg(n) \rfloor + 1$ (exactly as the worst successful case) if the tree is full; or it is either $h_n + 1$ (as the case of full tree) or h_n

4)Follows directly from the fact that a complete tree has the shortest length for a given number of nodes. ■



© Waleed A. Yousef 2008

Now, we will prepare some lemmas for calculating the average case

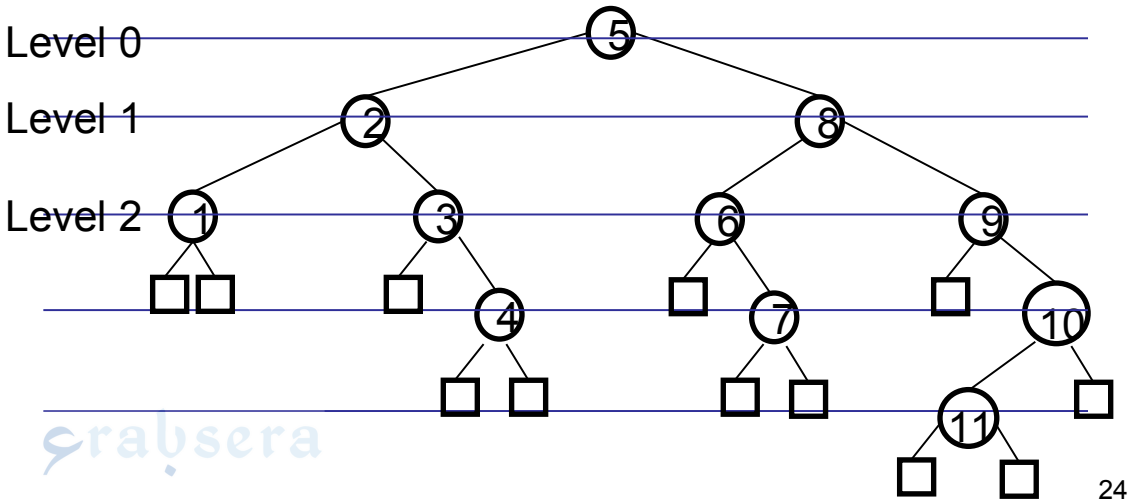
External node (or leaf), which corresponds to an unsuccessful search: The *external binary tree* is obtained by adding to each original node, 0, 1, or 2 children of a new distinct type (called external nodes or leaves) to make the outdegree of all the original nodes (now called internal) exactly equal to 2.

Lemma: An extended binary tree (not necessarily complete) on n internal vertices has $n + 1$ leaves.

Proof: Let the number of leaves be L_n (all are external by construction). Every internal node has two children (may be both internal, external, or one of each). Hence, the number of children is $2n$. Also, Every node in the tree, except the root is a child. Then the number of children is $L_n + n - 1$. Therefore:

$$L_n + n - 1 = 2n$$
$$L_n = n + 1$$

■



© Waleed A. Yousef 2008

Internal and External path lengths: The internal path length is the sum of path lengths of all internal nodes. Similarly, external path length is defined. For example, the internal path length in the figure is 0 + 1 + 1 + 2 + 2 + 2 + 3 + 3 + 3 + 4 = 21. The external path is 2+3 + 3 + 3 + 4 + 4 + 4 + 4 + 4 + 5 + 5 = 41.

Lemma: Let I_n and X_n be the internal and external path lengths of a binary tree of size n. Then

$$X_n = I_n + 2n.$$

Proof: We prove it by induction. The basis step at $n=1$ is obviously true, since $I_n=0$ and $X_n=2$. Assume that the statement is true for $n=k$. Now, remove a leaf external node and replace it with a leaf internal node along with its two external nodes; this adds a new external node. Now, the tree is of size $k+1$ and we have:

$$X_k = I_k + 2k. \quad \text{(by induction hypothesis)(1)}$$

$$X_{k+1} - 2(j+1) = X_k - j, \text{ and}$$

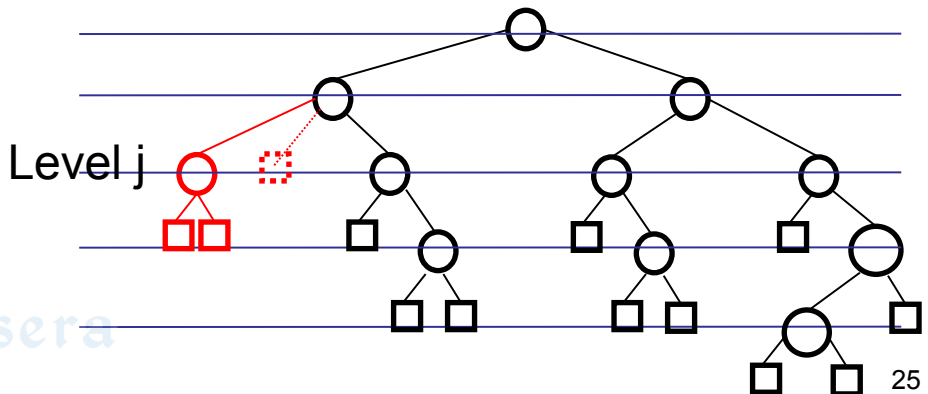
$$I_{k+1} - j = I_k .$$

Substituting back in (1) gives

$$X_{k+1} - 2(j+1) + j = I_{k+1} - j + 2k$$

$$X_{k+1} = I_{k+1} + 2(k+1)$$

■



© Waleed A. Yousef 2008

Lemma: The internal and external path lengths for a complete tree is given by:

$$I_n = (n+1)\lfloor \lg n \rfloor - 2^{\lfloor \lg n \rfloor + 1} + 2$$

$$X_n = (n+1)\left(\lfloor \lg n \rfloor + 2\right) - 2^{\lfloor \lg n \rfloor + 1}$$

Proof: Split I_n to p_1, the path of all saturated levels, and p_2, the path of the last level.

$$I_n = p_1 + p_2$$

$$p_1 = 0 \cdot 1 + 1 \cdot 2 + 2 \cdot 4 + 3 \cdot 8 + \cdots \left(h_n - 1\right) \cdot 2^{\left(h_n - 1\right)} = \sum_{i=1}^{h_n - 1} i2^i = 2\sum_{i=1}^{h_n - 1} i2^{i-1}$$

Because p_1 looks like a derivative of a quantity, consider

$$S\left(x\right) = 2\sum_{i=1}^{h_n - 1} x^i = 2\left(\sum_{i=0}^{h_n - 1} x^i - 1\right) = 2\left(\frac{x^{h_n} - 1}{x - 1} - 1\right) = 2\left(\frac{x^{h_n} - x}{x - 1}\right)$$

$$\frac{\partial S}{\partial x} = 2\sum_{i=1}^{h_n - 1} i x^{i-1} = 2\frac{\left(h_n x^{h_n - 1} - 1\right)\left(x - 1\right) - \left(x^{h_n} - x\right)}{\left(x - 1\right)^2}$$

$$p_1 = \left.\frac{\partial S}{\partial x}\right|_{x=2} = 2\left(h_n 2^{h_n - 1} - 2^{h_n} + 1\right)$$

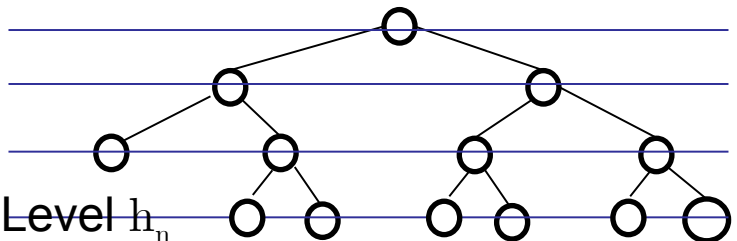
$$p_2 = h_n \cdot \left(n - \left(2^{h_n} - 1\right)\right)$$

$$I_n = \left(n+1\right)h_n - 2^{h_n + 1} + 2 = \left(n+1\right)\left\lfloor \lg n \right\rfloor - 2^{\left\lfloor \lg n \right\rfloor + 1} + 2$$

But for any tree (not necessarily complete)

$$X_n = I_n + 2n, \text{hence,}$$

$$X_n = \left(n+1\right)\left(\left\lfloor \lg n \right\rfloor + 2\right) - 2^{\left\lfloor \lg n \right\rfloor + 1}$$



© Waleed A. Yousef 2008

Theorem: The average successful case and unsuccessful case in binary search are given by:

$$C_n = \lfloor \lg n \rfloor + 1 - (2^{\lfloor \lg n \rfloor + 1} - \lfloor \lg n \rfloor - 2)/n$$
$$C'_n = \lfloor \lg n \rfloor + 2 - 2^{\lfloor \lg n \rfloor + 1}/(n+1)$$

Proof:

$$C_n = \sum_{i=1}^{n} \text{comparisons}_i \Pr(i) = \frac{1}{n}\sum_{i=1}^{n} \text{comparisons}_i = \frac{1}{n}\sum_{i=1}^{n}(1 + \text{InternalPath}_i) = \frac{1}{n}(n + I_n)$$

$$= 1 + \lfloor \lg n \rfloor - \frac{2^{\lfloor \lg n \rfloor + 1} - \lfloor \lg n \rfloor - 2}{n}$$

Similarily,because there are $n+1$ external nodes (unsuccessful search):

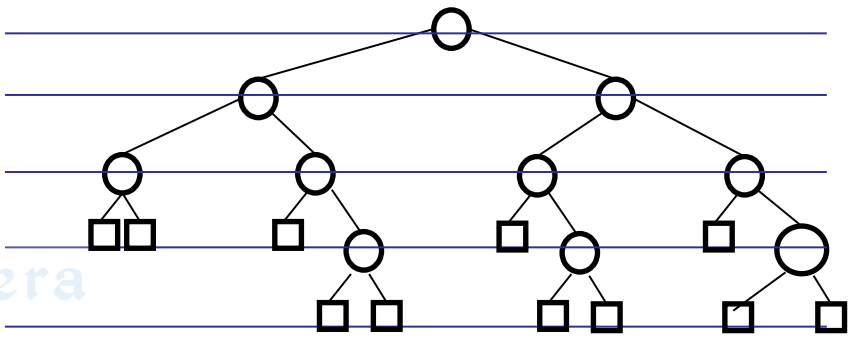
$$C'_n = \frac{1}{n+1}\sum_{i=1}^{n} \text{ExternalPath}_i = \lfloor \lg n \rfloor + 2 - \frac{2^{\lfloor \lg n \rfloor + 1}}{n+1}$$

■

Note:
$$\begin{aligned} C_n &= \lfloor \lg n \rfloor + 1 - (2^{\lfloor \lg n \rfloor + 1} - \lfloor \lg n \rfloor - 2)/n \\ &= \lfloor \lg n \rfloor + 1 - (2^{\lg n - \varepsilon + 1} - \lfloor \lg n \rfloor - 2)/n \\ &= \lfloor \lg n \rfloor + 1 - (n\, 2^{-\varepsilon + 1} - \lfloor \lg n \rfloor - 2)/n \\ &= \lfloor \lg n \rfloor + 1 - 2^{-\varepsilon + 1} + (\lfloor \lg n \rfloor + 2)/n, \end{aligned}$$

Which is $\Theta(\lg n)$

Similar treatment is immediate for C'_n



© Waleed A. Yousef 2008